

The forced-count graph lives on prefixes and suffixes

Notes toward a practical version of the polynomial-time SCS 2-approximation

October 8, 2026

Abstract

We study the count construction of Section 2 of *A Polynomial-Time 2-Approximation for Shortest Common Superstring* (OpenAI, September 2026), which is defined over all $O(L^2)$ distinct substrings of the input. We prove that its base graph is supported on the at most $2L$ prefixes and suffixes of the input strings, and that the periodicity rule may be restricted, without changing any count, to pairs of words that are each simultaneously a prefix and a suffix of input strings. Both statements follow from a single *shift lemma*. Finally, we give a closed form for the blocking sums in terms of the edge multiplicities, which yields an algorithm that never materializes the substring set. We also show that restricting the rule to least periods is *not* sound for this purpose. Finally, we implement the full connection phase of the preprint on top of these results. In experiments against an efficient greedy implementation, a simple order-merge post-pass makes the output competitive: it is shorter than greedy on repeat-rich inputs and on greedy's classical bad family, and it is often certified optimal by the lower bound W .

1 Setting

We use the notation of the preprint without change: the reduced instance $\mathcal{S}$ (substring-free, no empty or duplicate strings), the set V of its distinct substrings, the blocking functional $B_A(s; m)$ of (2.2), the counts m defined by the extension rules (2.4), the requirement rule, and the period rule (2.5), and the base multiplicities $u(s) = m(s) - \sum_c m(cs)$, $d(s) = m(s) - \sum_c m(sc)$. Missing words have count zero. Write

$$\lambda(s) = \sum_{c \in \Sigma} m(cs), \quad \rho(s) = \sum_{c \in \Sigma} m(sc),$$

so that $u(s) = m(s) - \lambda(s)$ and $d(s) = m(s) - \rho(s)$. Let $\mathcal{P}$ and $\mathcal{Q}$ be the sets of nonempty prefixes and nonempty suffixes of the strings in $\mathcal{S}$, and let $\mathcal{O} = \mathcal{P} \cap \mathcal{Q}$ be the set of *overlap words*. Clearly $|\mathcal{P}|, |\mathcal{Q}| \leq L$, and every required string lies in $\mathcal{O}$.

Definition 1.1. A *rule* is a quadruple (A, v, w, k) as in (2.5): A is a periodic text of period p , $v = A[0 : |v|] \in V$, w is a nonempty prefix of v , and $|v| = |w| + kp$ with $k \geq 1$. It is *triggered* if $m(v) > B_A(v; m)$; it then imposes $m(w) \geq B_A(w; m) + k + 1$, its *bound*. By construction, $m(s)$ is the maximum of $\lambda(s)$, $\rho(s)$, the requirement indicator, and the bounds of all triggered rules with $w = s$.

For a text A write $A^\leftarrow$ for its translate $A^\leftarrow(i) = A(i - 1)$. If $A[0 : |s|] = s$ and $a = A(-1)$, then $A^\leftarrow[0 : |s| + 1] = as$, so $B_{A^\leftarrow}(as; m)$ is defined. Translating a period- p text yields a period- p text.

Two facts about v and w in a rule are used repeatedly. Since w is a prefix of v , $w \notin \mathcal{P}$ implies $v \notin \mathcal{P}$. Since v has period p and $|v| - |w| = kp$, also $w = v[kp : |v|]$, so w is a suffix of v , and $w \notin \mathcal{Q}$ implies $v \notin \mathcal{Q}$.

2 Two elementary lemmas

Lemma 2.1 (Prefix-free sums). *Let y be a nonempty word and Z a finite prefix-free set of nonempty words. Then $\sum_{z \in Z} m(yz) \leq m(y)$. Symmetrically, if Z is suffix-free, then $\sum_{z \in Z} m(z y) \leq m(y)$.*

Proof. Induct on the maximum length in Z . Group Z by first letter c . If the one-letter word c lies in Z , prefix-freeness makes it the only member beginning with c . Otherwise $Z_c = \{z' : cz' \in Z\}$ is prefix-free, and induction gives $\sum_{z' \in Z_c} m(ycz') \leq m(y)$. Hence $\sum_{z \in Z} m(yz) \leq \sum_c m(y) \leq m(y)$ by the right extension rule. (If $y \notin V$, every term is zero.) The suffix-free case is the mirror image, using the left extension rule. $\square$

Lemma 2.2 (Bracket decomposition). *Let A be periodic with $A[0 : |s|] = s$, and put $a = A(-1)$, $e = A(|s|)$. Then*

$$B_A(s; m) = \Lambda_A(s) + B_{A^+}(as; m), \quad \Lambda_A(s) \leq \sum_{c \neq a} m(cs), \quad (2.1)$$

$$B_A(s; m) = R_A(s) + B_A(se; m), \quad R_A(s) \leq \sum_{d \neq e} m(sd), \quad (2.2)$$

where $\Lambda_A(s)$ counts, with multiplicity $m(r)$, the bracketing placements of s whose left mismatch is immediately before s , and $R_A(s)$ those whose right mismatch is immediately after s .

Proof. Consider a counted placement j of s in r as in (2.1)–(2.2).

Left. If $j \geq 2$, then position $j - 1$ is interior, so $r(j - 1) = A(-1) = a$ and as occurs at $j' = j - 1 \geq 1$. Substituting $A(i) = A^{\leftarrow}(i + 1)$ turns each condition of (2.1) for (A, s, j) into the corresponding condition for $(A^{\leftarrow}, as, j')$: the length condition $j' + |as| = j + |s| \leq |r| - 1$, the interior agreement $r(h) = A(h - j) = A^{\leftarrow}(h - j')$, and the two end mismatches $r(0) \neq A(-j) = A^{\leftarrow}(-j')$ and $r(|r| - 1) \neq A(|r| - 1 - j) = A^{\leftarrow}(|r| - 1 - j')$. The converse substitution is equally direct, so these placements are in bijection with the counted placements of as relative to $A^{\leftarrow}$, giving the second term. If $j = 1$, then $r = csxd$ with $c \neq a$, $x = A[|s| : |s| + |x|]$ and $d \neq A(|s| + |x|)$; the placement is unique for this r . For fixed c the tails xd form a prefix-free set, since two of them first differ where one leaves A . Lemma 2.1 bounds their total by $m(cs)$.

Right. Mirror image: a placement whose right mismatch is not adjacent to s has $r(j + |s|) = A(|s|) = e$ and is exactly a counted placement of se relative to A . The adjacent ones have the form $r = ysd$ with $d \neq e$, where the heads $y = cA[-|y| + 1 : 0]$, $c \neq A(-|y|)$, form a suffix-free set. $\square$

Corollary 2.3 (Monotonicity of the unblocked surplus). *Put $U_A(s) = m(s) - B_A(s; m)$. Then $U_A(s) \geq 0$, $U_A(s) \geq U_A(se)$ and $U_A(s) \geq U_{A^+}(as)$. In particular, for fixed A and w , the rules $(A, A[0 : |w| + kp], w, k)$ are triggered exactly for k up to some threshold, and only the largest triggered k gives a binding bound.*

Proof. By (2.2) and the extension rule, $m(s) \geq m(se) + \sum_{d \neq e} m(sd) \geq m(se) + R_A(s)$; subtract $B_A(s; m) = R_A(s) + B_A(se; m)$. The left inequality is the mirror image. Nonnegativity follows by induction on decreasing length, since $U_A(se) = 0$ when $se \notin V$. If $A[0 : |w| + kp] \in V$, so are its shorter prefixes, and the bound $B_A(w; m) + k + 1$ increases with k . $\square$

3 The shift lemma

Lemma 3.1 (Shift lemma). *Let (A, v, w, k) be a triggered rule.*

(a) *If $m(v) = \lambda(v)$, then $\lambda(w) \geq B_A(w; m) + k + 1$.*

(b) *If $m(v) = \rho(v)$, then $\rho(w) \geq B_A(w; m) + k + 1$.*

Proof. (a) Let $a = A(-1)$; it is the left neighbor in A of both v and w . By (2.1) for v and the trigger,

$$m(av) + \sum_{c \neq a} m(cv) = \lambda(v) = m(v) > B_A(v; m) = \Lambda_A(v) + B_{A^+}(av; m) \leq \sum_{c \neq a} m(cv) + B_{A^+}(av; m).$$

Hence $m(av) > B_{A^+}(av; m) \geq 0$, so $av \in V$. Now (A^+, av, aw, k) is a rule: A^+ has period p , $av = A^+[0 : |v| + 1) \in V$, aw is a nonempty prefix of av , and $|av| - |aw| = kp$. It is triggered, so the definition of m at the longer word aw gives $m(aw) \geq B_{A^+}(aw; m) + k + 1$. Applying (2.1) to w ,

$$\lambda(w) = m(aw) + \sum_{c \neq a} m(cw) \geq B_{A^+}(aw; m) + k + 1 + \Lambda_A(w) = B_A(w; m) + k + 1.$$

(b) Let $e = A(|v|)$. Since $|v| - |w| = kp$, also $e = A(|w|)$. By (2.2) for v and the trigger, exactly as in (a), $m(ve) > B_A(ve; m)$, so $ve \in V$. Then (A, ve, we, k) is a triggered rule: $ve = A[0 : |v| + 1)$, its prefix $A[0 : |w| + 1) = we$, and $|ve| - |we| = kp$. Hence $m(we) \geq B_A(we; m) + k + 1$, and (2.2) for w gives $\rho(w) = m(we) + \sum_{d \neq e} m(wd) \geq B_A(we; m) + k + 1 + R_A(w) = B_A(w; m) + k + 1$. $\square$

Remark 3.2. In words: if every counted occurrence of v extends to the left, then the $k + 1$ unblocked copies of w that a rule extracts from v can be shifted one letter to the left, as copies of aw in av . The rule is then already implied by the left extension inequality at w . Part (b) uses the extension to the right, together with the period: w and v have the same right neighbor e in A .

4 Main results

Theorem 4.1 (Prefix/suffix support). *For every nonempty $s \in V$:*

$$s \notin \mathcal{P} \implies m(s) = \lambda(s), \quad s \notin \mathcal{Q} \implies m(s) = \rho(s).$$

Consequently $u(s) > 0$ implies $s \in \mathcal{P}$, and $d(s) > 0$ implies $s \in \mathcal{Q}$. Every up edge of the base graph G joins two words of $\mathcal{P} \cup \{\varepsilon\}$, every down edge joins two words of $\mathcal{Q} \cup \{\varepsilon\}$, and G has at most $|\mathcal{P} \cup \mathcal{Q}| + 1 \leq 2L + 1$ vertices.

Proof. Induct on decreasing length, proving both implications together. Let $s \notin \mathcal{P}$. Then s is not required. For each letter d with $sd \in V$, also $sd \notin \mathcal{P}$, so induction gives $m(sd) = \lambda(sd)$. Therefore

$$\rho(s) = \sum_d \sum_c m(csd) = \sum_c \sum_d m(csd) \leq \sum_c m(cs) = \lambda(s),$$

by the right extension rule at each cs . Let (A, v, s, k) be a triggered rule with target s . Then $v \notin \mathcal{P}$ and $|v| > |s|$, so $m(v) = \lambda(v)$ by induction, and Lemma 3.1(a) bounds the rule by $\lambda(s)$. Every lower bound defining $m(s)$ is therefore at most $\lambda(s)$, and $m(s) = \lambda(s)$.

Let $s \notin \mathcal{Q}$. Symmetrically, $\lambda(s) \leq \rho(s)$, using induction on $cs \notin \mathcal{Q}$. For a triggered rule (A, v, s, k) , s is a suffix of v , so $v \notin \mathcal{Q}$. Induction gives $m(v) = \rho(v)$, and Lemma 3.1(b) bounds the rule by $\rho(s)$.

For the consequences, $u(s) = m(s) - \lambda(s)$ and $d(s) = m(s) - \rho(s)$. An up edge $\text{pref}(s) \rightarrow s$ is used only if $s \in \mathcal{P}$, and then $\text{pref}(s) \in \mathcal{P} \cup \{\varepsilon\}$. Similarly, a down edge $s \rightarrow \text{suf}(s)$ is used only if $s \in \mathcal{Q}$, and then $\text{suf}(s) \in \mathcal{Q} \cup \{\varepsilon\}$. $\square$

Theorem 4.2 (Overlap-word restriction). *Let m' be defined by the same descending recursion as m , but imposing (2.5) only for rules with $v \in \mathcal{O}$ and $w \in \mathcal{O}$. Then $m' = m$ on all of V ; in particular, both define the same base graph.*

Proof. Induct on decreasing length; assume $m' = m$ on all words longer than s . The quantities $\lambda(s)$, $\rho(s)$, $B_A(s; \cdot)$, and the triggers of all rules with target s depend only on longer words, so they coincide for m and m' . Since $m'(s)$ is the maximum of a subset of the bounds defining $m(s)$, we have $m'(s) \leq m(s)$. Conversely, consider a triggered rule (A, v, s, k) omitted from the restricted system, so $s \notin \mathcal{O}$ or $v \notin \mathcal{O}$. In either case $v \notin \mathcal{O}$, because $s \notin \mathcal{P}$ forces $v \notin \mathcal{P}$ and $s \notin \mathcal{Q}$ forces $v \notin \mathcal{Q}$. If $v \notin \mathcal{P}$, Theorem 4.1 gives $m(v) = \lambda(v)$ and Lemma 3.1(a) bounds the rule by $\lambda(s) \leq m'(s)$. If $v \notin \mathcal{Q}$, part (b) bounds it by $\rho(s) \leq m'(s)$. Thus $m(s) \leq m'(s)$. $\square$

Corollary 4.3. *Every count is determined by the counts on $\mathcal{P} \cup \mathcal{Q}$: for $s \notin \mathcal{P}$, $m(s) = \sum_c m(cs)$, and for $s \notin \mathcal{Q}$, $m(s) = \sum_c m(sc)$. The only period rules that must be evaluated are those with $v, w \in \mathcal{O}$, and, by Corollary 2.3, for each overlap word w and each compatible periodic text A only the largest triggered k .*

5 Least periods do not suffice

The rule (2.5) ranges over all periods of v , not only its least period. Restricting the rule to least periods weakens the bound, even on overlap words.

Example 5.1. Let $\mathcal{S} = \{\text{aabaa}, \text{baaab}\}$, with optimum 7 (baaabaa). The word $v = \text{aabaa} \in \mathcal{O}$ has least period 3 and also period 4. With $A = (\text{aaba})^\infty$, the rule $(A, v, \mathbf{a}, 1)$ is triggered and gives $m(\mathbf{a}) \geq B_A(\mathbf{a}; m) + 2 = 2 + 2 = 4$. All other bounds on $m(\mathbf{a})$ are at most 3. With all periods, $W = 6$; with least periods only, $m(\mathbf{a}) = 3$ and $W = 5$. The base-graph lower bound is therefore genuinely weaker under the least-period restriction, and any efficient implementation must handle all periods of overlap words.

6 Empirical confirmation and what remains

A direct implementation of the preprint's Section 2 confirmed every statement above, with no violations: Theorem 4.1 and both parts of Lemma 3.1 (about 5×10^6 triggered rules), Corollary 2.3, and Theorem 4.2 (identical base graphs on 4×10^5 random, periodic, and Fibonacci-type instances).

The following observation is not proved here.

Conjecture 6.1. *Without the period rule, $W = \sum_c m(c)$ equals the weight of a minimum cycle cover of the overlap (distance) graph of $\mathcal{S}$.*

It held on all 1500 instances tested. If true, the extension-only part of the construction is computable in $O(L)$ time by known Aho–Corasick techniques, and only the periodic correction is new.

Consequence for the algorithm. Theorems 4.1 and 4.2 reduce the vertex set from $O(L^2)$ to $|\mathcal{P} \cup \mathcal{Q}| \leq 2L$, and the period rules to pairs of overlap words. On 600 reads of length 100 from a random genome this is 1.25×10^6 substrings versus 1.1×10^5 prefixes and suffixes and 2,879 overlap

words, with the number of (v, p) rule sources falling from about 4×10^5 to 944. The blocking sums are handled in the next section.

7 Blocking sums from the edge multiplicities

The blocking functional $B_A(w; m)$ ranges over bracketing words that need not lie in $\mathcal{P} \cup \mathcal{Q}$. We now express it, and the trigger of a rule, using only u and d , which by [Theorem 4.1](#) live on $\mathcal{P}$ and $\mathcal{Q}$ respectively and have total multiplicity W each.

Throughout, A is a periodic text with $A[0 : |x|) = x$, and we write $x_n = A[0 : n)$ for $n \geq |x|$.

Lemma 7.1 (Left-bracket telescoping). *For a nonempty word y whose left context is read in A (that is, we put $y_\mu = A[-\mu : 0) y$), define*

$$\Lambda_A^*(y) = \sum_{\mu \geq 0} \sum_{c \neq A(-\mu-1)} m(c y_\mu),$$

the total count of words obtained from y by extending along A to the left and then adding one mismatching letter. Then $\Lambda_A^(y) = m(y) - \sum_{\mu \geq 0} u(y_\mu)$.*

Proof. For each μ , $\sum_c m(c y_\mu) = \lambda(y_\mu) = m(y_\mu) - u(y_\mu)$, and the term with $c = A(-\mu-1)$ is $m(y_{\mu+1})$. Hence the μ -th inner sum is $m(y_\mu) - u(y_\mu) - m(y_{\mu+1})$. Summing over μ telescopes. The sum is finite, since $y_\mu \notin V$ for large μ , and then all terms vanish. $\square$

Theorem 7.2 (Closed form for blocking sums). *Put*

$$D_A(x) = \sum_{n \geq |x|} d(x_n), \quad T_A(x) = \sum_{n \geq |x|} \sum_{c \neq A(n)} \sum_{\mu \geq 0} u(A[-\mu : n) c).$$

Then the unblocked surplus is

$$U_A(x) = m(x) - B_A(x; m) = D_A(x) + T_A(x), \tag{7.1}$$

and equivalently

$$B_A(x; m) = \rho(x) - (D_A(x) - d(x)) - T_A(x). \tag{7.2}$$

Proof. In [Lemma 2.2](#), the bracketing placements of x_n whose right mismatch is adjacent are exactly the words $c' A[-\mu : 0) x_n c$ with $c' \neq A(-\mu-1)$ and $c \neq A(n)$, each with one placement. Hence, with equality, $R_A(x_n) = \sum_{c \neq A(n)} \Lambda_A^*(x_n c)$. Applying [Lemma 7.1](#) to $y = x_n c$, and using $\sum_{c \neq A(n)} m(x_n c) = \rho(x_n) - m(x_{n+1}) = m(x_n) - d(x_n) - m(x_{n+1})$,

$$R_A(x_n) = m(x_n) - m(x_{n+1}) - d(x_n) - \sum_{c \neq A(n)} \sum_{\mu \geq 0} u(A[-\mu : n) c).$$

Iterating the right decomposition $B_A(x_n) = R_A(x_n) + B_A(x_{n+1})$ from $n = |x|$ until $x_n \notin V$, where everything vanishes, gives $B_A(x; m) = m(x) - D_A(x) - T_A(x)$. Finally $m(x) - d(x) = \rho(x)$. $\square$

Remark 7.3 (Walk interpretation). On the closed walks of Sections 2–3 of the preprint, [\(7.1\)](#) says that an occurrence of x is unblocked exactly when, before any mismatch has entered on both sides, either the walk deletes the first letter of the occurrence while its window is still an A -aligned word x_n (the term D_A), or the walk appends a first right mismatch c while its window is an A -aligned word covering x (the term T_A). This matches the proof of [Lemma 3.4](#) of the preprint.

Corollary 7.4 (Counts from the supports of u and d). *Process the words of $\mathcal{P} \cup \mathcal{Q}$ in decreasing length. For each such word s ,*

$$\lambda(s) = \sum_{y \text{ longer, ends with } s} u(y), \quad \rho(s) = \sum_{y \text{ longer, starts with } s} d(y),$$

by the telescoping identity in the proof of Lemma 3.1 of the preprint. Set $m(s) = \rho(s)$ if $s \notin \mathcal{Q}$ and $m(s) = \lambda(s)$ if $s \notin \mathcal{P}$. If $s \in \mathcal{O}$, take the maximum of $\lambda(s)$, $\rho(s)$, the requirement indicator, and, for each pending rule (A, k) ,

$$\rho(s) + k + 1 - (D_A(s) - d(s)) - T_A(s).$$

After fixing $m(s)$, set $u(s) = m(s) - \lambda(s)$ and $d(s) = m(s) - \rho(s)$. Then, for $s \in \mathcal{O}$ and each period p of s , the rule with A generated by $s[0 : p)$ is triggered iff $D_A(s) + T_A(s) > 0$. Every quantity used involves u and d only on words already processed, together with $d(s)$ itself in the trigger.

In particular a rule can raise $m(w)$ above $\rho(w)$ only if $k + 1 > (D_A(w) - d(w)) + T_A(w)$.

Lemma 7.5 (Primitive templates suffice). *If $v[0 : p) = t^j$ with $j \geq 2$, every rule (A, v, w, k) with period p is dominated by the rule (A, v, w, jk) with period $|t|$.*

Proof. Both rules use the same text $A = t^\infty$, hence the same trigger and the same $B_A(w; m)$. Since $|v| - |w| = kp = (jk)|t|$, the second rule exists, and its bound $B_A(w; m) + jk + 1$ is larger. $\square$

Implementation and validation. A prototype (`compact.py`) keeps u in a forward and a reversed weighted trie and d in a forward trie, built incrementally in decreasing length. Then λ and ρ are subtree sums, and D_A is a single walk along A . For T_A , words $A[-\mu : n)c$ with the same start residue modulo p are the same string, so $T_A(x)$ takes p walks, each weighted by the number of admissible shifts. The prototype never builds V . On 4.6×10^5 random, periodic and Fibonacci-type instances it reproduced the reference implementation's u and d exactly. It processes 2×10^4 reads of length 100 ($L = 2 \times 10^6$) in under 90s in pure Python, whereas the reference construction would have to enumerate about 2×10^7 distinct substrings there and evaluate blocking sums over them. The remaining costs are $O(|s|)$ trie navigation per word and the T_A walks. Both should become amortized constant time per step using Aho–Corasick node identifiers and Fenwick trees over Euler tours.

8 The connection phase in practice

The results above make the count construction practical. We now implement the remainder of the preprint (Sections 3–7: layers, group processing, cycle opening and the Euler tour) on top of the compact counts and compare the output with the classical greedy algorithm.

Implementation. The prototype (`connect.py`) follows the preprint step by step. It decomposes the base graph into closed walks and lifts each walk to windows of its periodic text. It then groups walks by primitive text and builds the ordered layers of Lemma 3.3 by sorting normalized exit ends, checking that the edge multiset is unchanged. Next it processes the groups as in Section 5, which covers incoming host requests (Lemma 4.3), the baseline, the easy cases, the threshold search and the record search, followed by the collective, individual and request cases. It then opens the cycles of planned links as in Section 6 and finally extracts the superstring by an Euler tour from ε . Every operation is materialized as an explicit closed walk of hierarchical-graph edges via the ordered-window construction of Lemma 4.1. Each inequality that the preprint proves for an operation is asserted at run time, as are balance of every added walk, the total budget $\leq W$, and the containment of every input in the output. The substring set V is never built. Word membership

in V is never queried either, because every prescribed window is guaranteed by the preprint to be an actual word.

Validation. On 6.4×10^4 random, periodic and Fibonacci-type instances no assertion failed. On every instance with at most 7 strings, the output was within a factor 2 of the exact optimum, the worst ratio being 1.9375. Coverage-guided mutation fuzzing (about 25 minutes on 60 cores, with no failures) exercised host requests, collective and individual blocks, easy band joins, and the opening of cycles of length at least 2. Six branches were never reached: rooting layers below a baseline (both cases), self-cycles (both steps), records on a different phase of the same group, and hosts receiving requests from more than one child group. Their run-time checks are in place but untested.

Lemma 8.1 (Order-merge). *Let T be a common superstring of a substring-free family $\mathcal{S}$. Order $\mathcal{S}$ by the position of its leftmost occurrence in T and merge consecutive strings with maximum overlap. The result is a common superstring of length at most $|T|$.*

Proof. Let the leftmost occurrences be $[p_i, e_i)$ with $p_1 < \dots < p_n$. Since $\mathcal{S}$ is substring-free, also $e_1 < \dots < e_n$. If $e_i > p_{i+1}$, the interval $[p_{i+1}, e_i)$ spells a proper suffix of s_i that is a proper prefix of s_{i+1} , so the maximum overlap o_i is at least $e_i - p_{i+1}$. Hence $|s_{i+1}| - o_i \leq e_{i+1} - e_i$ in that case. Otherwise $|s_{i+1}| - o_i \leq e_{i+1} - p_{i+1} \leq e_{i+1} - e_i$. Summing, the merged length is at most $|s_1| + e_n - e_1 = e_n - p_1 \leq |T|$. $\square$

Applied to the output of the 2-approximation, order-merge therefore preserves the guarantee $|T| \leq 2W \leq 2 \text{ OPT}$. It costs $O(L)$ with an Aho–Corasick scan of T .

Experiments. The baseline is the maximum-overlap greedy algorithm in its efficient form (`greedy.py`). An Aho–Corasick automaton enumerates all suffix–prefix overlaps through failure links. The overlaps are then processed in decreasing length, with skip pointers over strings that already have a predecessor and a union–find structure that forbids cycles. On 2×10^4 small instances its output length agreed with a naive pairwise-merge greedy except for tie-breaking. Because $W \leq \text{OPT}$, the ratio $|T|/W$ certifies an upper bound on the true approximation ratio, and $|T| = W$ certifies optimality. Table 1 lists representative large instances, and the full suite is in `experiments.py`.

family	instance	n	W	ours	ours+om	greedy
random-genome reads	cov=1 l=50	394	12114	1.0025	1.0000	1.0000
	cov=1 l=100	198	12033	1.0018	1.0001	1.0000
	cov=3 l=50	1163	19010	1.0025	1.0000	1.0000
	cov=3 l=100	592	18835	1.0044	1.0031	1.0000
	cov=10 l=50	3648	19996	1.0025	1.0000	1.0000
	cov=10 l=100	1912	19983	1.0048	1.0000	1.0000
repeat-rich reads	s0 cov=3 l=100	589	16950	1.0143	1.0031	1.0060
	s0 cov=10 l=100	1816	18023	1.0159	1.0000	1.0034
	s1 cov=3 l=100	571	16533	1.0240	1.0000	1.0018
	s1 cov=10 l=100	1749	17340	1.0130	1.0000	1.0138
	s2 cov=3 l=100	580	16924	1.0165	1.0042	1.0008
	s2 cov=10 l=100	1784	18094	1.0150	1.0001	1.0032
microsatellite reads	unit=2 cov=5 l=40	438	3498	1.0234	1.0060	1.0040
	unit=3 cov=5 l=40	553	4127	1.0482	1.0148	1.0216
	unit=5 cov=5 l=40	697	4587	1.1195	1.0275	1.0541
random strings	$ \Sigma =2$ n=300 l=12	291	1098	1.0191	1.0055	1.0000
	$ \Sigma =4$ n=300 l=12	300	2505	1.0040	1.0000	1.0016
	$ \Sigma =2$ n=1000 l=20	1000	9810	1.0016	1.0006	1.0000
greedy’s bad family	k=5, 20 copies	60	280	1.1464	1.0000	1.5714
	k=20, 20 copies	60	880	1.0466	1.0000	1.8636
	k=50, 20 copies	60	2080	1.0197	1.0000	1.9423

Table 1: Output length relative to the lower bound W (smaller is better; 1 means provably optimal). Bold marks the shorter of ours+om and greedy when they differ. “Ours” is the raw Euler-tour output, “ours+om” adds order-merge. Genomes have length 2×10^4 ; reads are error-free.

Over all 21 large instances, ours+om was shorter than greedy on 11, longer on 6, and tied on 4. It attained W , and was therefore provably optimal, on 11 instances, against 8 for greedy. The comparison by family is as follows.

- *Error-free reads from a random genome.* The base graph is nearly connected, and the connection phase adds at most 0.5% of W . Greedy is optimal or near-optimal here and slightly ahead.
- *Repeat-rich genomes and microsatellites.* Ours+om is shorter than greedy on 7 of 9 instances and reaches W on 3 of them. These are exactly the periodic structures the count rule (2.5) is designed for.
- *Greedy’s classical bad family* $\{c(ab)^k, (ba)^k, (ab)^k e\}$, in 20 copies over disjoint alphabets: greedy’s ratio tends to 2 (1.94 at $k = 50$), while ours+om is exactly optimal.
- *Small instances against the exact optimum* (200 periodic and 200 binary instances, 3–7 strings). The raw output is wasteful, with mean ratio 1.15 and 1.21, because every connection is paid for by round trips through ε . After order-merge the mean ratios are 1.002 and 1.013, against 1.003 and 1.006 for greedy. Ours+om is optimal on 194 and 168 of the 200 instances, greedy on 192 and 180.

Cost. The Python prototype is 10–80 times slower than greedy, for example 11.9s against 0.23 s on 1,816 repeat-rich reads. The main costs are the $O(|s|)$ trie navigation of [Corollary 7.4](#), the T_A walks, and the record search of Section 5, which scans all $2W$ layer windows for each hard case. All three admit standard indexed implementations.